IBVAP — Intrusion Detection Logic & Event Generation

05. Spatial & Temporal Perimeter Breach Rules

---

1. Mathematical & Spatial Logic of Intrusion

An **intrusion** in IBVAP is defined mathematically as the spatial intersection of a tracked object's bounding box centroid (or base coordinates) with a user-defined 2D polygon vector representing a restricted border perimeter.

Spatial Rule Equation

Let $P_{restricted} = \{(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)\}$ be the closed polygon vertices of the restricted zone.

Let $B_i(t) = [x_{min}, y_{min}, x_{max}, y_{max}]$ be the bounding box of person Track ID i at frame t.

The centroid $C_i(t)$ is calculated as:

$$C_i(t) = \left(\frac{x_{min} + x_{max}}{2}, \frac{y_{min} + y_{max}}{2} \right)$$

An intrusion condition $I_i(t)$ is satisfied if:

$$I_i(t) = \begin{cases} 1 & \text{if } C_i(t) \in \text{Polygon}(P_{restricted}) \\ 0 & \text{otherwise} \end{cases}$$

State Transition Logic

- `INITIAL_INSIDE`: Target is detected inside the restricted polygon at the very first frame of tracking.
- `ENTER`: Target transitions from outside $P_{restricted}$ ($I_i(t-1) = 0$) to inside ($I_i(t) = 1$). This fires an Intrusion Breach Event (`EVT-XXX`).
- `EXIT`: Target transitions from inside $P_{restricted}$ ($I_i(t-1) = 1$) to outside ($I_i(t) = 0$).

---

2. Concrete Project Examples (Verified from Source)

The prototype codebase (`output/day3_zone_events.csv` & `output/secure_audit_log.csv`) documents the following verified breach examples:

Example 1: Intrusion Breach EVT-001

- **Track ID:** `Track 2`
- **Frame Number:** `Frame 30`
- **Timestamp:** `00:01.20`
- **Event Type:** `Zone Entry` (`ENTER`)
- **Risk Level:** `HIGH`
- **Evidence Snapshot:** `output/evidence/EVT-001_track_2_frame_30.jpg`
- **Flow:** Person Track 2 approaches border polygon $\rightarrow$ Centroid crosses polygon boundary at Frame 30 $\rightarrow$ `ENTER` event triggered $\rightarrow$ Snapshot captured $\rightarrow$ EVT-001 written to database & SHA-256 audit log.

Example 2: Intrusion Breach EVT-002

- **Track ID:** `Track 7`
- **Frame Number:** `Frame 108`
- **Timestamp:** `00:04.33`
- **Event Type:** `Zone Entry` (`ENTER`)
- **Risk Level:** `HIGH`
- **Evidence Snapshot:** `output/evidence/EVT-002_track_7_frame_108.jpg`
- **Flow:** Person Track 7 moves towards perimeter fence $\rightarrow$ Enters restricted area at Frame 108 $\rightarrow$ `ENTER` event triggered $\rightarrow$ Snapshot captured $\rightarrow$ EVT-002 logged with previous SHA-256 hash reference.

Example 3: Intrusion Breach EVT-003

- **Track ID:** `Track 36`
- **Frame Number:** `Frame 177`
- **Timestamp:** `00:07.10`
- **Event Type:** `Zone Entry` (`ENTER`)
- **Risk Level:** `HIGH`
- **Evidence Snapshot:** `output/evidence/EVT-003_track_36_frame_177.jpg`
- **Flow:** Person Track 36 breaches inner border corridor at Frame 177 $\rightarrow$ `ENTER` event triggered $\rightarrow$ Snapshot captured $\rightarrow$ EVT-003 appended to SHA-256 audit chain.

---

3. False-Positive Reduction & Duplicate Prevention

- **Track-Based Deduplication:** Because ByteTrack assigns a persistent `person_track_id`, an ongoing intrusion by the same person does not spam hundreds of duplicate event logs. The breach event (`ENTER`) is triggered once upon zone transition.
- **Track Memory Buffer (`track_buffer: 60`):** If a person is temporarily obscured by a vehicle or terrain feature for up to 60 frames (2 seconds at 30 FPS), ByteTrack retains their existing Track ID instead of creating a false new track ID and firing a false duplicate alert.
- **Risk Classification Engine:** Assigns `HIGH` risk to restricted zone breaches, while reserving `MEDIUM` or `LOW` for perimeter proximity or unconfirmed zone activity.